



PARADIGMAS DE PROGRAMACIÓN

TP N°2

Universidad Abierta Interamericana

**LICENCIATURA EN PRODUCCIÓN DE SIMULADORES Y
VIDEOJUEGOS**

Damián Carbone

19/05/2025



El asset pack elegido será:
Ninja adventure Asset pack.

<https://pixel-boy.itch.io/ninja-adventure-asset-pack>

Concepto general del juego:

El proyecto tratará sobre un juego de acción y combate Top-down 2D, donde el jugador controlará un ninja retirado que vive tranquilamente en su casa, alejado del pueblo. Un día descubre que toda su comida fue robada. Al parecer, diversos monstruos del bosque fueron los responsables. A pesar de estar retirado, decide que no puede permitirle que se salgan con las suyas y se pone a la acción para recuperarla y vengarse.

El prototipo tendrá un único nivel jugable ambientado en un bosque cercano al pueblo. El jugador deberá avanzar por el escenario, derrotar enemigos, recolectar monedas y comida, encontrar mejoras permanentes y utilizar distintos pergaminos de habilidad para enfrentarse al Rey Monstruo, jefe final del nivel.

Durante el recorrido, el jugador podrá encontrar pergaminos que le permitirán recordar antiguas técnicas ninja. El jugador solo podrá tener un pergamino equipado a la vez, por lo que al recoger uno nuevo dejará caer el anterior. Esto permitirá intercambiar habilidades según la situación, como lanzar fuego, lanzar una roca o realizar un dash de viento.

El objetivo principal del prototipo será llegar hasta la guarida del Rey Monstruo, derrotarlo y recuperar la comida robada. Como consecuencia secundaria, el jugador también terminará salvando al pueblo de la invasión de monstruos.

Durante el recorrido, el jugador interactuará con los siguientes elementos:

- Enemigos básicos: criaturas del bosque que perseguirán al jugador y atacarán cuerpo a cuerpo.
- Enemigos a distancia: monstruos que atacarán lanzando proyectiles.
- Objetos destructibles: cajas o arbustos que podrán romperse y soltar monedas o comida.
- Coleccionables: monedas para aumentar el puntaje y comida para recuperar vida.
- Mejoras permanentes: objetos especiales que aumentarán las capacidades del ninja (salud, daño de ataque, cooldown de habilidad especial).
- Pergaminos: objetos que permiten intercambiar la habilidad especial equipada del ninja.
- Jefe final: el Rey Monstruo, un enemigo más resistente que marcará el final del proto.

Para esta entrega, el juego será principalmente lineal, aunque podrá tener pequeños desvíos para representar la idea de exploración.



Mecánicas:

El jugador se mueve en las 4 direcciones con WASD y da un golpe con J.

El jugador podrá obtener diversas mejoras permanentes a través de los items.

Las mejoras posibles serán:

- Comida especial: aumenta la vida máxima del jugador, daño del ataque principal, velocidad de movimiento o tiempo de recuperación de la habilidad especial.
- Pergamino de técnica: otorga una habilidad especial del ninja.

Habilidad del pergamino:

Los pergaminos funcionan como habilidades especiales intercambiables, elegidas principalmente por conveniencia o preferencia del jugador.

Con la tecla K activamos la habilidad de pergamino, y luego para volver a usarla tendremos que esperar un cooldown.

Pergaminos:

Pergamino de Fuego: permite lanzar una bola de fuego a distancia. Tarda en lanzarse pero tiene un área grande.

Pergamino de Viento: permite hacer un dash rápido que corta a los enemigos al paso.

Pergamino de Roca: permite sacar una pesada del suelo que impacta a los enemigos cercanos. Es muy rápida y hace mucho daño, pero tiene un área de acción pequeña.

Pergamino de Hielo: Un proyectil que congela al enemigo que impacta.



Conceptos Fundamentales de POO:

El código fuente del proyecto estará organizado de esta forma:

Clases y Objetos:

Cada elemento importante del juego será representado mediante una clase específica. Por ejemplo, el jugador será controlado por una clase `PlayerController`, los enemigos tendrán clases como `EnemyBase`, `MeleeEnemy`, `RangedEnemy` y `BossEnemy`, mientras que los objetos recolectables tendrán clases como `CoinCollectible`, `FoodCollectible`, `UpgradeCollectible` y `SkillCollectible`.

Encapsulamiento:

Los atributos importantes del juego, como la vida del jugador, el daño del ataque, la velocidad de movimiento, el cooldown de las habilidades y la cantidad de monedas, estarán protegidos para evitar modificaciones directas desde otros scripts. Estos valores serán modificados mediante métodos controlados, como recibir daño, curarse, aumentar monedas o aplicar una mejora permanente.

Abstracción:

Se utilizarán clases base para representar comportamientos generales. Por ejemplo, una clase base `EnemyBase` contendrá la lógica común de todos los enemigos, como la vida, la detección del jugador, la recepción de daño y el comportamiento básico de ataque. De esta forma, cada enemigo específico solo deberá definir sus características particulares.

Herencia:

Los enemigos específicos heredarán de la clase base `EnemyBase`. Por ejemplo, el enemigo cuerpo a cuerpo, el enemigo a distancia y el Rey Monstruo compartirán una misma estructura general, pero cada uno tendrá sus propios valores y formas de atacar. Esto permite reutilizar código y evitar repetir la misma lógica en cada enemigo.

Polimorfismo:

Los enemigos podrán tener un método de ataque general que será redefinido según el tipo de enemigo. El enemigo cuerpo a cuerpo atacará al acercarse al jugador, el enemigo a distancia lanzará proyectiles y el jefe final tendrá ataques más fuertes o patrones especiales. Aunque todos sean enemigos, cada uno responderá de manera diferente al mismo comportamiento base.

Métodos con Parámetros y Retornos:

Los sistemas del juego se comunicarán mediante métodos que reciban datos y devuelvan resultados cuando sea necesario. Por ejemplo, el jugador podrá recibir una cantidad específica de daño, curarse con cierta cantidad de vida, aumentar su daño de ataque o consultar si tiene una habilidad especial equipada. Esto permitirá que los sistemas sean más ordenados y fáciles de reutilizar.



Implementación de interfaces:

Se utilizarán interfaces para desacoplar comportamientos y permitir que objetos distintos puedan compartir una misma funcionalidad sin depender de una misma clase padre.

IDamageable:

Esta interfaz será implementada por todos los objetos que puedan recibir daño, como el jugador, los enemigos, el jefe final y los objetos destructibles del escenario. Gracias a esta interfaz, cualquier ataque del juego podrá dañar un objeto sin necesidad de saber exactamente qué tipo de objeto es. Por ejemplo, el golpe del jugador, una bola de fuego, una roca o un proyectil enemigo solo deberán detectar si el objeto golpeado puede recibir daño.

ICollectible:

Esta interfaz será implementada por todos los objetos que el jugador pueda recoger, como monedas, comida, mejoras permanentes y pergaminos. Cada coleccionable tendrá un efecto distinto: las monedas aumentarán el puntaje, la comida recuperará vida, las mejoras permanentes aumentarán estadísticas del personaje y los pergaminos permitirán intercambiar la habilidad especial equipada.

Uso de los pergaminos con interfaces:

Los pergaminos funcionarán como objetos recolectables, pero tendrán un comportamiento particular: al recoger uno, el jugador equipará esa habilidad especial y soltará el pergamino que tenía anteriormente. Esto permitirá que el jugador cambie entre habilidades como fuego, viento, roca o hielo sin usar un inventario complejo.



Sistemas y Buenas Prácticas:

Para mantener el proyecto organizado y fácil de ampliar, se aplicarán algunas buenas prácticas de desarrollo.

Scriptable Objects para datos:

Se podrán utilizar Scriptable Objects para guardar datos de personajes, enemigos, mejoras y pergaminos. Por ejemplo, la vida máxima, el daño, la velocidad, el cooldown. el tipo de habilidad o el sprite asociado a cada item podrían configurarse desde archivos de datos, separando la información de la lógica del código.

Sistema simple de habilidades:

Las habilidades de pergamino estarán organizadas mediante una estructura común. Todas serán habilidades especiales, pero cada una tendrá un efecto diferente: fuego lanzará un proyectil con área, viento realizará un dash ofensivo, roca golpeará una zona cercana y hielo congelará al enemigo impactado. Esto permitirá agregar nuevas habilidades en el futuro sin modificar todo el sistema del jugador.

Escalabilidad del proyecto:

Aunque el prototipo tendrá un solo nivel, la estructura quedará preparada para poder agregar más enemigos, más pergaminos, nuevas mejoras, más mapas o nuevos jefes en el futuro sin tener que rehacer los sistemas principales.